AI Applications in Research

Claude Code as a Research Partner for Empirical Social Science

Prepared for the "Scott Cunningham newsletter" routine

27 July 2026

Abstract

This note summarises the main concepts behind using AI agents — and **Claude Code** in particular — in empirical research. It synthesises themes developed across Scott Cunningham's *Scott's Mixtape* Substack "Claude Code" essay series, framing the shift from writing code to *directing* an agent, the economics of near-zero-cost cognitive output, the failure modes this creates, and the disciplined "harness" researchers build to keep agent-assisted work correct and reproducible.

Contents

1 What changes when an agent enters the workflow 1

2 The economics: falling marginal cost, rising verification cost 1

3 The defining failure mode: AI agent drift 2

4 The harness: a rigid, staged checklist 2

5 Verification as a first-class step 3

6 Managing abundance: the canon and the reading load 3

7 The human question: do mistakes still matter? 3

8 Key takeaways 4

Sources & further reading 4

1 What changes when an agent enters the workflow

An *agentic* coding assistant such as Claude Code is qualitatively different from a chat model. It runs in a loop — reading files, running code (Python, R, Stata), inspecting output, and editing — so the researcher increasingly *specifies intent* rather than typing every line. The practical consequences:

- **Machine time and human time become near substitutes.** Tasks that once cost hours of a researcher's attention can be delegated, so the same person can attempt far more analyses in the same calendar time.
- **The unit of work shifts from "code" to "instruction."** The scarce skill becomes clearly specifying the estimand, the sample, and the checks — not recalling syntax.
- **Output becomes abundant.** Tables, figures, robustness checks, and even draft prose can be produced faster than a human can carefully read them.

2 The economics: falling marginal cost, rising verification cost

A recurring frame in the series is a simple production story for *cognitive output* — a regression table, a figure, a paragraph, or a whole worked manuscript.

- **Marginal cost of production collapses.** Where producing a unit of output once cost MC_0 (human attention and time), agents push the *private* marginal cost toward $MC_1 \approx \varepsilon$.
- **Private vs. social cost diverge.** Cheap private production does not imply cheap social cost: if verification lags production, errors can accumulate in the aggregate scientific record.
- **Verification is the new bottleneck.** Because output is generated rather than hand-built, the binding constraint moves from *making* results to *trusting* them. The researcher's comparative advantage shifts toward specification, judgement, and verification.

A useful way to see the aggregate-error point: even if an agent's per-task error rate is *lower* than a human's, decoupling production from verification plus a large increase in the *quantity* of output can still raise the total number of errors entering the literature, depending on how elastic output is to cost.

3 The defining failure mode: AI agent drift

The series names a characteristic hazard — **agent drift** — where an assisted project silently wanders away from the analysis the researcher believes they are running:

- **Silent sample changes** ($N \to N+k$): a stray or stale cleaning step quietly alters the analytical sample, so estimates change for reasons the author never authored.
- **Code and file proliferation**: near-duplicate scripts, figures, and "fixed" versions multiply until it is unclear which artifacts are canonical.
- **Growing distance from the work**: the more the agent produces, the less first-hand contact the researcher has with each step — exactly the contact that once caught mistakes.

This is the modern echo of a classic hazard: a single stale line in a cleaning file can flip a result, and it is easiest to miss precisely when you did not type it yourself.

4 The harness: a rigid, staged checklist

The proposed remedy is not to trust the agent less *ad hoc*, but to build a **harness** — an explicit, staged workflow that the human designs and the agent executes within. For causal-inference projects the harness is a checklist of locked stages, roughly:

```
PKG → TARGET → BITE → COVARIATES → ROLLOUT → OUTCOME → POWER → ESTIMATOR → EVENT
    → FALSIFY → MECHANISM → PIPELINE → DELIVER → SIGN
```

Each stage is defined before estimation and is "locked" once agreed, so the agent cannot silently revisit an earlier decision. Supporting devices include:

- **A status dashboard** — a colour-coded view of which stages are open, locked, or stale, so drift is visible at a glance.
- **Custom skills / slash-commands** — e.g. `/covariates`, `/referee2`, `/blindspot` — that package a repeatable audit or step.
- **Hooks and locks** — automated guards that fire on defined events (e.g. a changed sample size) to force a human check.
- **Design-first / pre-registered estimand** — fixing the target parameter and identification strategy *before* the agent produces numbers.

5 Verification as a first-class step

Because verification is the scarce good, the series treats it as its own workflow, not an afterthought:

- **Author/reviewer separation**: use one agent (or pass) to produce and a separate, adversarial pass to check — a "`Referee 2`" protocol that writes a formal referee report and demands a revise-and-resubmit.
- **Cross-language replication**: re-implement the same estimator across Stata, R, and Python (and across packages, e.g. multiple Callaway–Sant'Anna implementations) and require the numbers to agree.
- **Blind-spot and falsification checks**: deliberately search for what the main specification hides — placebo/event-study leads, alternative covariate sets, and sensitivity to the sample definition.
- **Read the diff, not the vibe**: review the actual code and output changes an agent makes rather than trusting a fluent summary of them.

6 Managing abundance: the canon and the reading load

Two late-series essays address the *consequences* of cheap output rather than its production:

- **Stale code, deprecation, and the canon.** With agents generating many artifacts, the researcher must deliberately *curate* which code and exhibits are canonical (enter the replicable pipeline) and *deprecate* the rest — treating a project like a maintained canon rather than an ever-growing pile.

- **Am I reading too much?** Abundant output can pull a researcher toward low-value, low-comprehension reading. The fix is to manage finite attention: prefer concision, shift modalities (figures over prose, summaries over raw dumps), and stay at the high-value part of the task.

7 The human question: do mistakes still matter?

The most recent essay (*Which research mistakes matter for human capital?*, 27 July 2026) turns from mechanics to epistemics. It weighs two ledgers:

- **Output value.** Catching coding errors before publication is good in aggregate — for every caught error, many uncaught ones reach the record — so an agent that surfaces errors is a clear plus.
- **Human-capital value.** Yet the author's most durable lessons — folder hygiene, pipelined code, version control, and above all *scientific integrity* — came *from* making and suffering a real mistake. As with poker, you learn because there are stakes.

The essay deliberately reaches no verdict: we want accuracy (that is why we wear seat belts), but a workflow that prevents every mistake may forfeit the failure-driven knowledge that classrooms and mentorship cannot supply. It is an open tension for anyone adopting agents in research, not a solved problem.

8 Key takeaways

1. Claude Code turns the researcher into a *director and verifier* of analysis rather than a line-by-line producer.
2. The marginal cost of producing results falls toward zero; the scarce input becomes *verification and judgement*.
3. The signature risk is *drift* — silent sample changes and artifact proliferation that move a project away from its intended analysis.
4. Discipline comes from a *harness*: locked, staged checklists, status dashboards, custom skills, and hooks that make drift visible and force human checks.
5. Verification deserves its own adversarial workflow — author/reviewer separation, cross-language replication, falsification, and diff-level review.
6. Abundance must be *curated* (a canon) and *read selectively*, because attention, not output, is now the constraint.
7. Whether removing the ability to make mistakes is entirely good remains an open, genuinely unresolved question about how researchers accumulate skill.

Sources & further reading

- Scott Cunningham, *Scott's Mixtape* Substack, "Claude Code" series, `https://causalinf.substack.com/s/claude-code`. Essays synthesised here include *Claude Code 56: Falling marginal costs of production, output, and aggregate errors*; *Am I reading too much?* (17 Jul 2026); *Stale*

code, deprecation, and the canon (21 Jul 2026); and *Which research mistakes matter for human capital?* (27 Jul 2026).

- `scunning1975/MixtapeTools` — public repository of Claude Code templates, the `Referee 2` audit protocol, a PDF-reading skill, and a `CLAUDE.md` session template. https://github.com/scunning1975/mixtapetools
- Scott Cunningham, *Causal Inference: The Mixtape* (Yale University Press), https://mixtape.scunning.com/.

Concept summary compiled from publicly described themes of the newsletter series; figures and quotations are paraphrased for summary purposes.